

Проектирование микропроцессорной системы на базе МК i8051

Описание программной части

Листов 28

Инв. № подл.	Подп. и дата
Взам. инв. №	Инв. № дубл.
Подп. и дата	Подп. и дата

Выполнил: Давыдова Е.П.
Группа: А-09-23
Вариант: 2

Проверил: Ключанский А.А.

Москва, 2026

АННОТАЦИЯ

В данном документе приведено описание программы «Программа управления объектом», предназначенной для управления объектом микропроцессорной системой (МПС) на базе микроконтроллера AT89S53: ввода данных с клавиатуры и от внешнего устройства, формирования управляющих сигналов Y_1 и Y_2 и вывода информации на семисегментный индикатор.

Для функционирования данной программы необходима МПС, собранная по схеме электрической принципиальной варианта 2 (пояснительная записка по аппаратной части).

Исходным языком программы «Программа управления объектом» является ассемблер микроконтроллеров семейства MCS-51.

Основными функциями программы являются: инициализация МПС, сканирование клавиатуры, работа с кольцевым буфером на 1520 байт, выработка stroba Y_1 длительностью 205 мс, выработка сигнала Y_2 со стробом 1869 мкс, индикация на одном разряде.

Оформление программного документа «Описание программы» произведено по требованиям ЕСПД (ГОСТ 19.101-77, ГОСТ 19.103-77, ГОСТ 19.104-78, ГОСТ 19.105-78, ГОСТ 19.106-78, ГОСТ 19.402-78, ГОСТ 19.604-78).

СОДЕРЖАНИЕ

1. Общие сведения.....	4
2. Функциональное назначение.....	5
2.1. Классы решаемых задач.....	5
2.2. Назначение программы.....	5
2.3. Сведения о функциональных ограничениях на применение.....	5
3. Описание логической структуры.....	6
3.1. Алгоритм программы.....	6
3.2. Структура программы с описанием функций составных частей и связи между ними.....	18
3.3. Связи программы с другими программами.....	19
4. Входные данные.....	20
4.1. Клавиатура.....	20
4.2. Сигнал X2.....	20
5. Выходные данные.....	21
5.1. Характер и организация выходных данных.....	21
5.2. Формат, описание и способ кодирования выходных данных.....	21
ПЕРЕЧЕНЬ ПРИНЯТЫХ СОКРАЩЕНИЙ.....	23
ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ.....	24
ПРИЛОЖЕНИЕ А. Листинги программ.....	25

1. ОБЩИЕ СВЕДЕНИЯ

Обозначение и наименование программы — «Программа управления объектом», вариант 2, группа А-09-23.

Программа низкого уровня не требует для своей работы иных программ. Исходным языком программирования является ассемблер MCS-51 (синтаксис Intel ASM51, совместимый с Keil A51 и MCU 8051 IDE).

Среда разработки и инструментальные средства: интегрированная среда Keil μ Vision V5 (Keil Software, пакет PK51); ассемблер — A51; компоновщик — BL51.

2. ФУНКЦИОНАЛЬНОЕ НАЗНАЧЕНИЕ

2.1. Классы решаемых задач

- инициализация МПС;
- сканирование клавиатуры — ввод сигнала X_1 (число, введённое с клавиатуры);
- формирование сигнала Y_1 (строб заданной длительности);
- ввод сигнала X_2 (сохранение данных в кольцевом буфере);
- формирование сигнала Y_2 и сопровождающего строба;
- вывод символа на индикатор.

2.2. Назначение программы

МПС принимает информацию $\{X\}$ об объекте управления от датчиков, вырабатывает управляющие воздействия $\{Y\}$ в соответствии с заданным алгоритмом управления и подаёт их на исполнительные механизмы. Программа разбита на подзадачи по функциональному признаку:

- инициализация МПС;
- обработка прерываний: опрос клавиатуры, запись в буфер, таймеры генераторов сигналов Y ;
- определение кода нажатой клавиши;
- вывод данных на семисегментный индикатор;
- формирование сигнала Y_2 ;
- обеспечение работы кольцевого буфера (запись и чтение отсчётов X_2).

В соответствии с вариантом (табл. 1 ТЗ) разработан код трёх программ:

- программа 1 — обработчик прерывания клавиатуры (сканирование матрицы), код нажатой клавиши в аккумуляторе или 0FFh;
- программа 2 — чтение из кольцевого буфера (данные через аккумулятор) с управлением флагами пустоты и переполнения;
- программа 3 — расчёт и выдача сигнала Y_2 со стробом длительностью T_2 (прерывание таймера).

2.3. Сведения о функциональных ограничениях на применение

В качестве основы для построения МПС выбран микроконтроллер семейства MCS-51 — AT89S53, тактовая частота 12 МГц (машинный цикл 1 мкс). Таймер 2 и сторожевой таймер не используются (совместимость с i8051).

Интерфейсы:

- входные данные X_2 — 8-разрядное целое без знака по независимой параллельной шине со стробом записи низкого уровня в двухпортовое ОЗУ, нерегулярно, до 4 отсчётов в секунду;
- входные данные X_1 — десятичная цифра 0–9 с матричной клавиатуры 4×3 (сканирование по столбцам);
- выходной сигнал Y_2 — 8-разрядный параллельный код в выходном регистре со стробом низкого уровня длительностью 1869 мкс на выводе P1.3;
- выходной сигнал Y_1 — строб низкого уровня длительностью 205 мс на выводе P1.2;
- индикация — один семисегментный индикатор (общий анод), время индикации символа не более $T_3 = 2,380$ с.

3. ОПИСАНИЕ ЛОГИЧЕСКОЙ СТРУКТУРЫ

3.1. Алгоритм программы

После сброса МК выполняет инициализацию (рисунок 3.1): указатель стека ставится на 07h, разрешается дешифратор адреса (CS_EN = 1), буфер объявляется пустым («голова» и «хвост» на 8000h), стробы Y1 и Y2 переводятся в пассивную единицу, на все столбцы клавиатуры выдаются нули, индикатор гасится, оба таймера переводятся в 16-разрядный режим и разрешаются прерывания. Далее работает основной цикл (рисунок 3.2): пока X1 не равно нулю, в буфере есть отсчёты и предыдущий строб Y2 уже закончился, программа берёт из буфера самый старый X2, считает Y2 и выдаёт его со стробом; если X1 = 0, а на выходе ещё стоит ненулевой Y2, один раз выдаётся ноль.

Всё, что связано с внешними событиями и длительностями, вынесено в прерывания. По INT0 (нажатие клавиши) опрашивается клавиатура и выполняется реакция на клавишу: цифра становится новым X1 и показывается на индикаторе, «Тест» запускает строб Y1, «Сброс» очищает буфер (рисунки 3.3, 3.4). По INT1 (строб внешнего устройства) отсчёт X2 с разъёма заносится в ячейку «головы» (рисунки 3.5, 3.6). Timer0 тикает каждые 5 мс и отсчитывает T1 = 205 мс для строба Y1 и T3 = 2,38 с для гашения индикатора (рисунок 3.10), Timer1 одним отсчётом задаёт длительность строба Y2 = 1869 мкс и снимает его в своём обработчике (рисунок 3.13). Сам основной цикл ничего не ждёт, поэтому новое нажатие или отсчёт обрабатываются сразу.

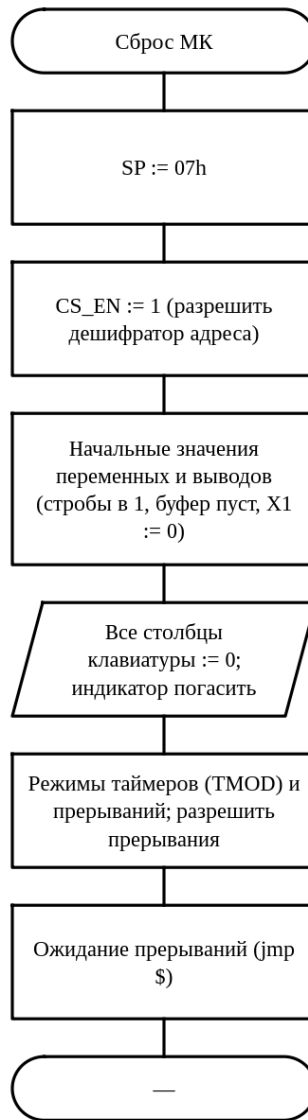


Рис. 3.1. Схема алгоритма: Инициализация МПС и основная программа

Схема «Инициализация МПС и основная программа» выполняется так: вход — «Сброс МК»; $SP := 07h$; $CS_EN := 1$ (разрешить дешифратор адреса); начальные значения переменных и выводов (стробы в 1, буфер пуст, $X1 := 0$); обмен с устройством: все столбцы клавиатуры := 0; индикатор погасить; режимы таймеров (TMOD) и прерываний; разрешить прерывания; ожидание прерываний (jmp \$).

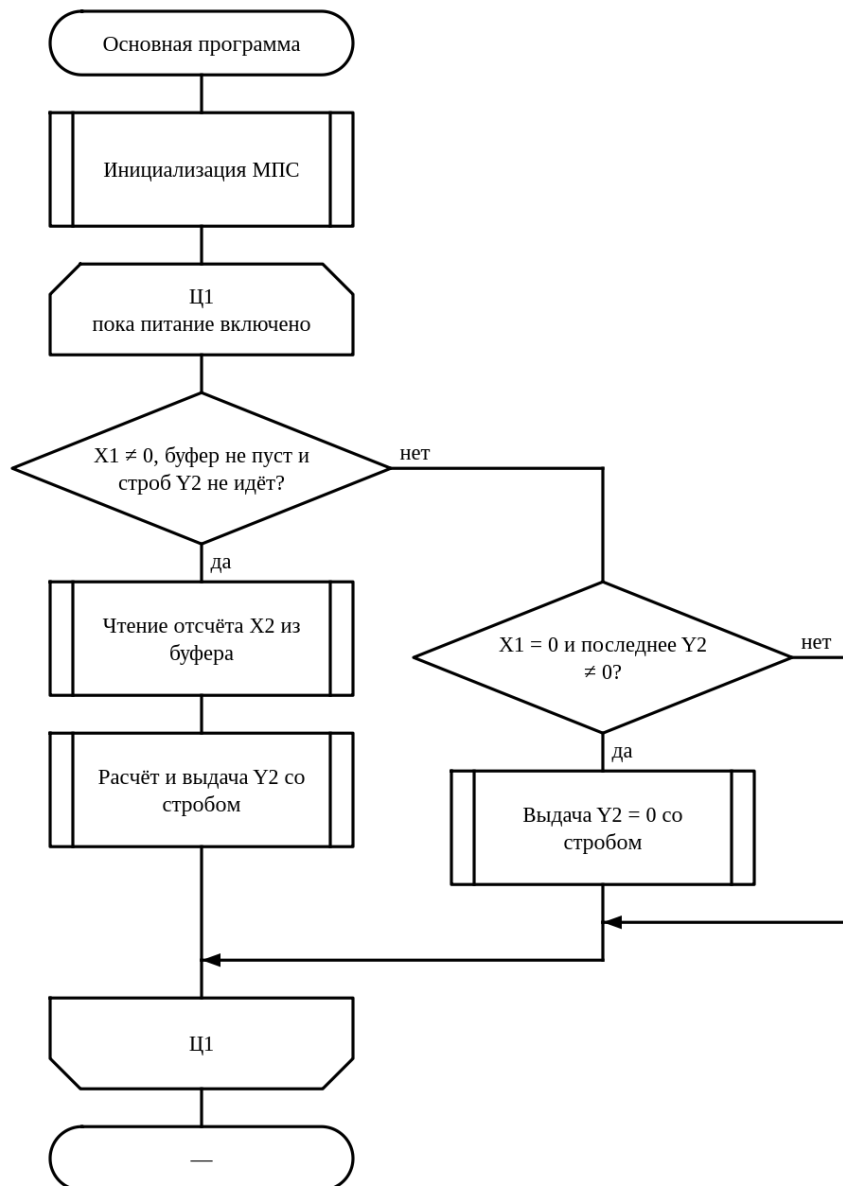


Рис. 3.2. Схема алгоритма: Основной цикл программы

Схема «Основной цикл программы» выполняется так: вход — «Основная программа»; вызывается подпрограмма: инициализация МПС; в цикле Ц1 (пока питание включено) повторяется: проверяется условие « $X1 \neq 0$, буфер не пуст и строб Y2 не идёт»: если да — вызывается подпрограмма: чтение отсчёта X2 из буфера, вызывается подпрограмма: расчёт и выдача Y2 со стробом, иначе — проверяется условие « $X1 = 0$ и последнее Y2 $\neq 0$ »: если да — вызывается подпрограмма: выдача Y2 = 0 со стробом.

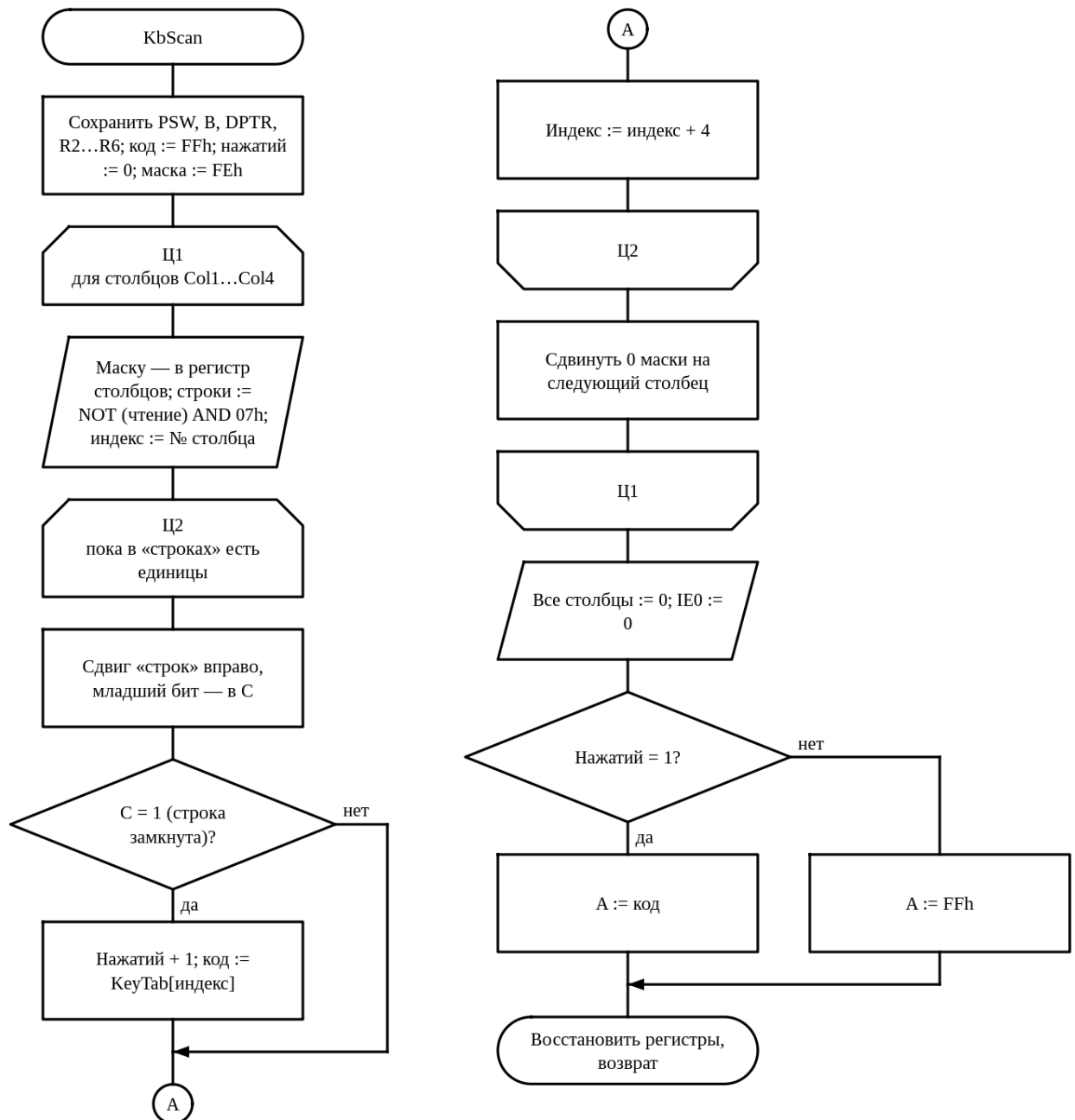


Рис. 3.3. Схема алгоритма: Опрос клавиатуры KbScan (обработчик INT0)

Процедура KbScan сначала сохраняет используемые регистры, затем по очереди опускает в ноль один из четырёх столбцов, а остальные держит в единице. Прочитанные строки инвертируются и маскируются тремя младшими битами, так что единица означает замкнутый контакт. Внутренний цикл сдвигает эти биты по одному: на каждый замкнутый контакт увеличивается счётчик нажатий и из таблицы KeyTab берётся код клавиши, индекс в которой равен «строка $\times$ 4 + столбец». После перебора на все столбцы снова выдаётся ноль, чтобы следующее нажатие опять вызвало INT0, и сбрасывается флаг IE0 от спадов во время опроса. Код возвращается, только если замкнут ровно один контакт, иначе — 0FFh.

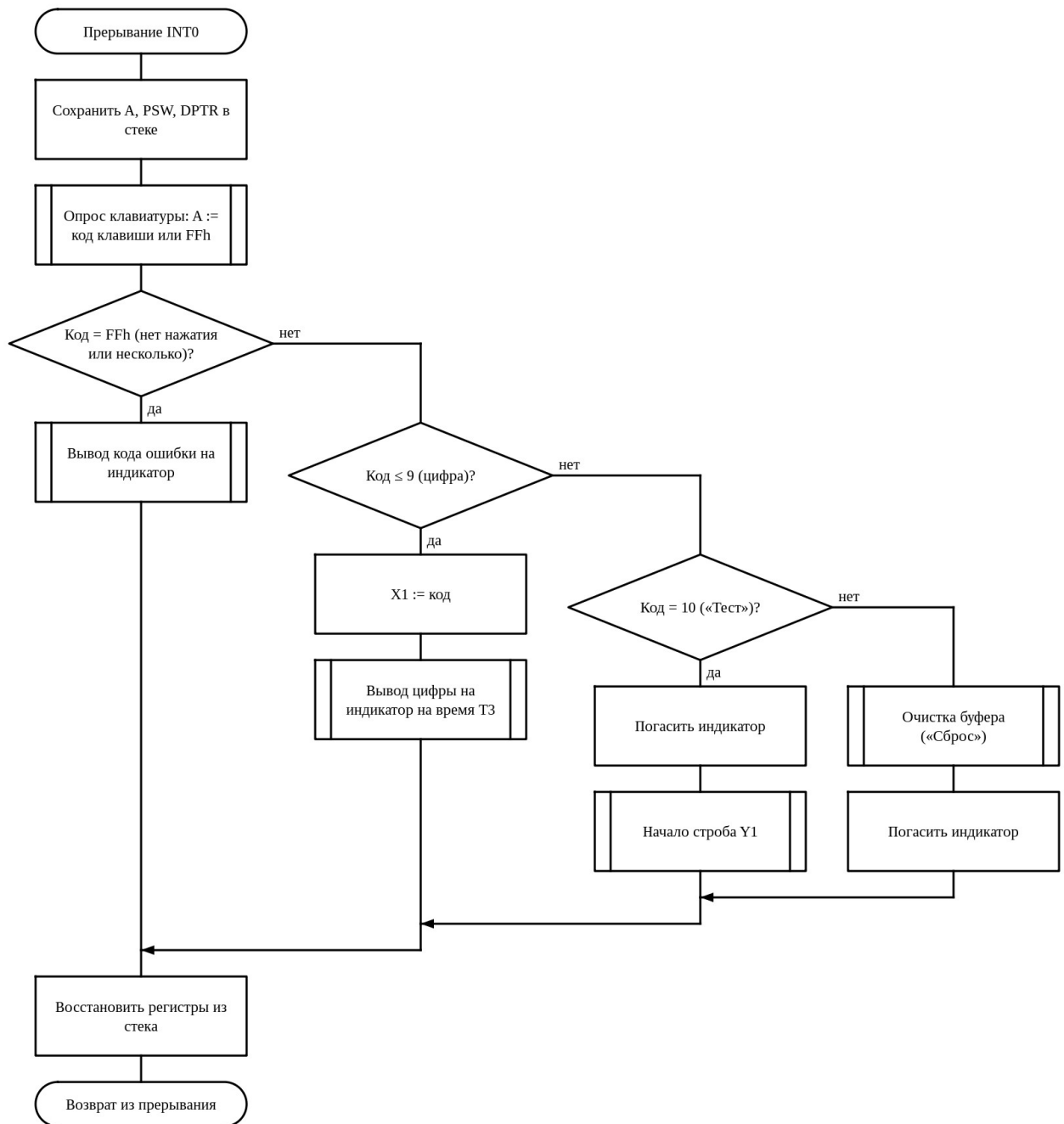


Рис. 3.4. Схема алгоритма: Обработчик прерывания INT0: реакция на клавишу

Схема «Обработчик прерывания INT0: реакция на клавишу» выполняется так: вход — «Прерывание INT0»; сохранить A, PSW, DPTR в стеке; вызывается подпрограмма: опрос клавиатуры: A := код клавиши или FFh; проверяется условие «Код = FFh (нет нажатия или несколько)»: если да — вызывается подпрограмма: вывод кода ошибки на индикатор, иначе — проверяется условие «Код ≤ 9 (цифра)»: если да — X1 := код, вызывается подпрограмма: вывод цифры на индикатор на время T3, иначе — проверяется условие «Код = 10 («Тест»)»: если да — погасить индикатор, вызывается подпрограмма: начало строба Y1, иначе — вызывается подпрограмма: очистка буфера («Сброс»), погасить индикатор; восстановить регистры из стека; в конце — возврат из прерывания.

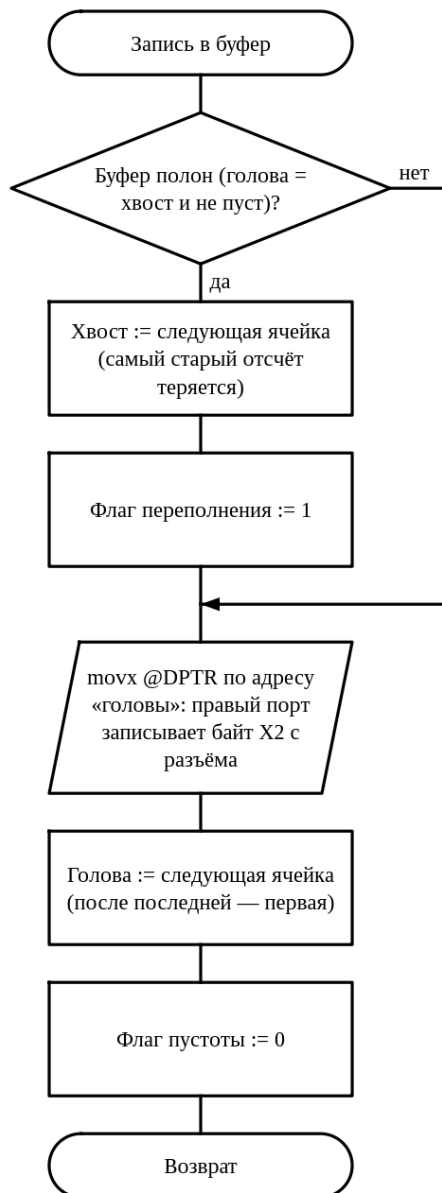


Рис. 3.5. Схема алгоритма: Запись отсчёта в кольцевой буфер

Схема «Запись отсчёта в кольцевой буфер» выполняется так: вход — «Запись в буфер»; проверяется условие «Буфер полон (голова = хвост и не пуст)»: если да — хвост := следующая ячейка (самый старый отсчёт теряется), флаг переполнения := 1; обмен с устройством: movx @DPTR по адресу «головы»: правый порт записывает байт X2 с разъёма; голова := следующая ячейка (после последней — первая); флаг пустоты := 0; в конце — возврат.

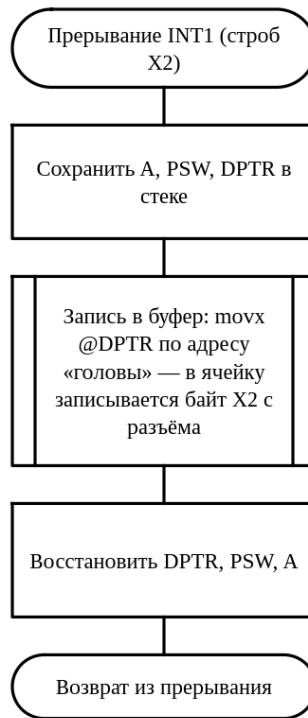


Рис. 3.6. Схема алгоритма: Обработчик прерывания INT1: приём отсчёта X2 от внешнего устройства

Схема «Обработчик прерывания INT1: приём отсчёта X2 от внешнего устройства» выполняется так: вход — «Прерывание INT1 (строб X2)»; сохранить A, PSW, DPTR в стеке; вызывается подпрограмма: запись в буфер: `movx @DPTR` по адресу «головы» — в ячейку записывается байт X2 с разъёма; восстановить DPTR, PSW, A; в конце — возврат из прерывания.

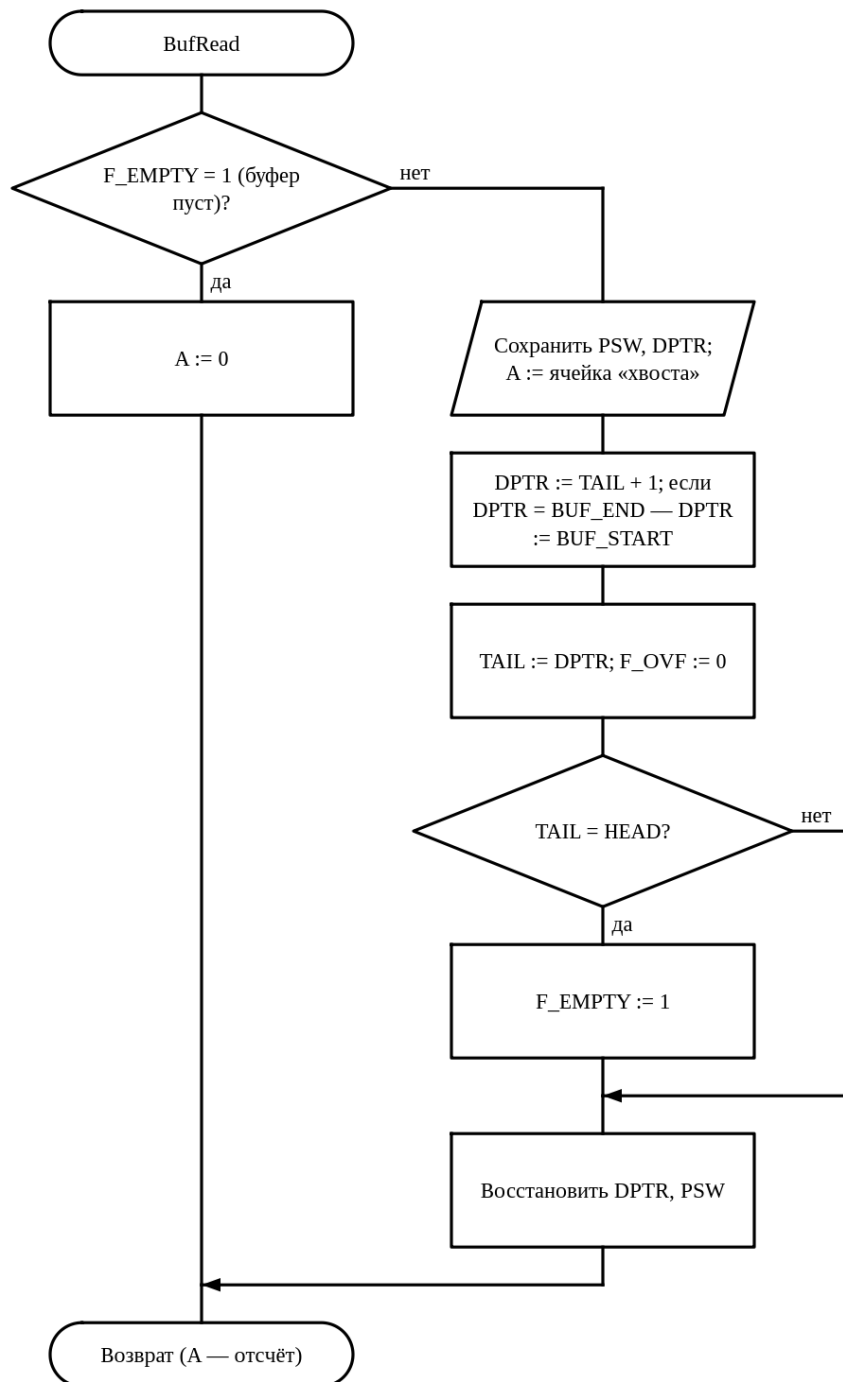


Рис. 3.7. Схема алгоритма: Чтение отсчёта из кольцевого буфера BufRead

BufRead сначала проверяет флаг пустоты: если буфер пуст, возвращается $A = 0$, а указатели и флаги не меняются. Иначе по адресу «хвоста» читается самый старый отсчёт, «хвост» сдвигается на следующую ячейку, а после последней ячейки 85EFh переходит на 8000h (сравниваются оба байта адреса). Раз одна ячейка освободилась, флаг переполнения сбрасывается. Если «хвост» догнал «голову», прочитан последний отсчёт и ставится флаг пустоты.



Рис. 3.8. Схема алгоритма: Очистка кольцевого буфера (клавиша «Сброс»)

Схема «Очистка кольцевого буфера (клавиша «Сброс»)» выполняется так: вход — «Очистка буфера»; запретить прерывание INT1; хвост := голова; флаг пустоты := 1; флаг переполнения := 0; разрешить прерывание INT1; в конце — возврат.

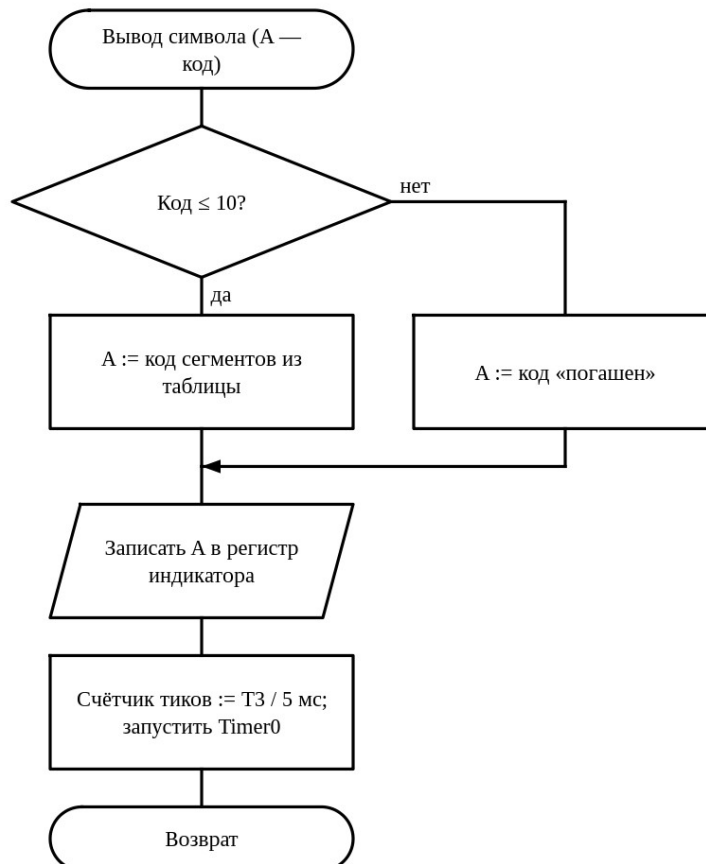


Рис. 3.9. Схема алгоритма: Вывод символа на индикатор

Схема «Вывод символа на индикатор» выполняется так: вход — «Вывод символа (A — код)»; проверяется условие «Код ≤ 10 »: если да — A := код сегментов из таблицы, иначе — A := код «погашен»; обмен с устройством: записать A в регистр индикатора; счётчик тиков := T3 / 5 мс; запустить Timer0; в конце — возврат.

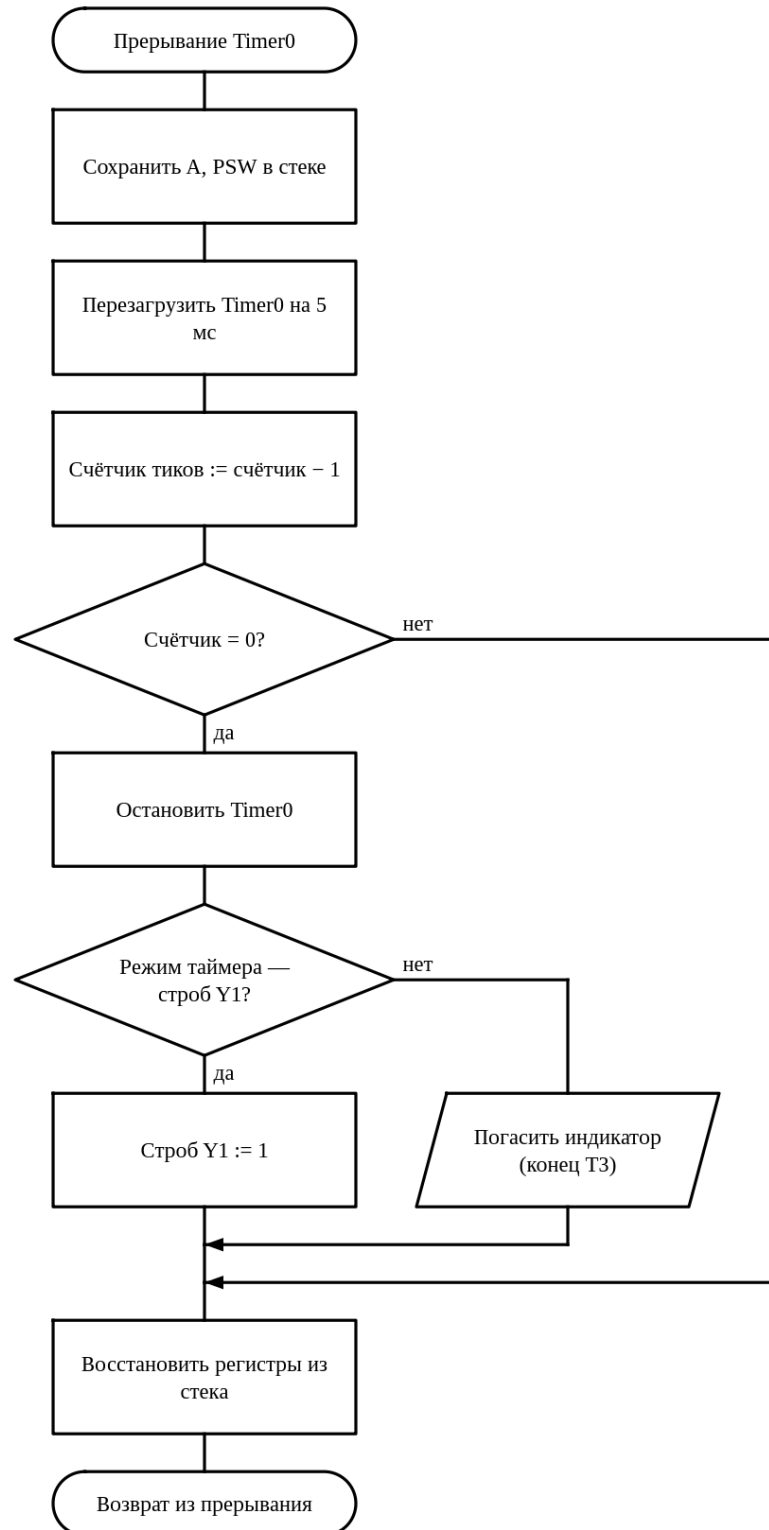


Рис. 3.10. Схема алгоритма: Обработчик прерывания Timer0: отсчёт T1 и T3

Схема «Обработчик прерывания Timer0: отсчёт T1 и T3» выполняется так: вход — «Прерывание Timer0»; сохранить A, PSW в стеке; перезагрузить Timer0 на 5 мс; счётчик тиков := счётчик - 1; проверяется условие «Счётчик = 0»: если да — остановить Timer0, проверяется условие «Режим таймера — строб Y1»: если да — строб Y1 := 1, иначе — обмен с

устройством: погасить индикатор (конец ТЗ); восстановить регистры из стека; в конце — возврат из прерывания.

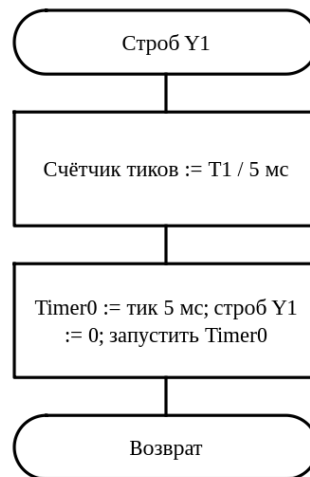


Рис. 3.11. Схема алгоритма: Начало строба Y1

Схема «Начало строба Y1» выполняется так: вход — «Строб Y1»; счётчик тиков := T1 / 5 мс; Timer0 := тик 5 мс; строб Y1 := 0; запустить Timer0; в конце — возврат.

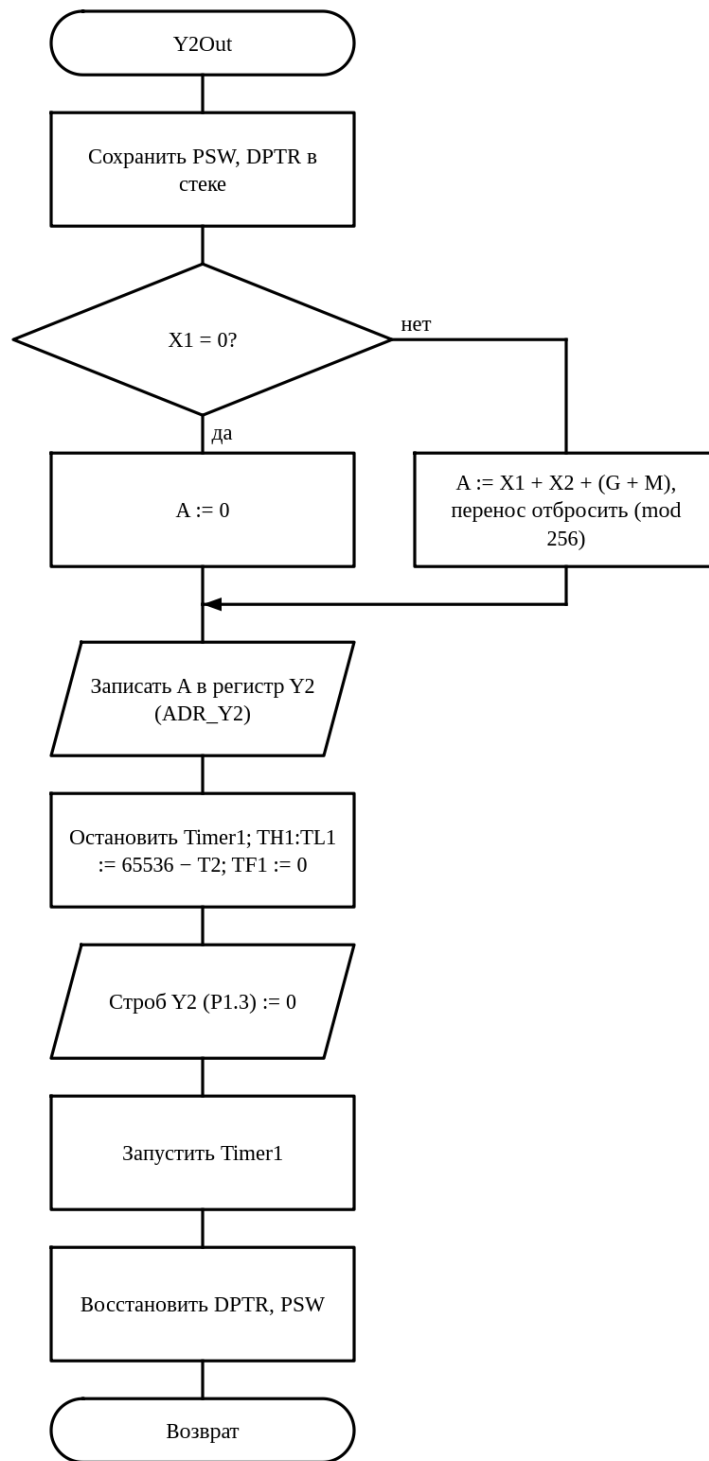


Рис. 3.12. Схема алгоритма: Расчёт и выдача сигнала Y2 (Y2Out)

Y2Out при $X1 = 0$ выдаёт ноль, иначе складывает $X1$, $X2$ и константу $G + M = 11$; перенос из старшего разряда просто отбрасывается, что и даёт остаток от деления на 256. Результат сначала записывается в регистр Y2, и только после этого на выводе P1.3 начинается строб: так внешнее устройство к началу строба уже видит новое значение. Timer1 загружается на 1869 мкс и запускается, а процедура сразу возвращается — конец строба формирует прерывание Timer1 (рисунок 3.13).



Рис. 3.13. Схема алгоритма: Обработчик прерывания Timer1: конец строба Y2

Схема «Обработчик прерывания Timer1: конец строба Y2» выполняется так: вход — «Прерывание Timer1»; остановить Timer1; строб Y2 := 1; в конце — возврат из прерывания.

3.2. Структура программы с описанием функций составных частей и связи между ними

Программа состоит из инициализации, основной программы (ожидание прерываний) и подпрограмм, описанных ниже. Описание каждой подпрограммы взято из заголовочного комментария в исходном тексте (приложение А).

3.2.1 Подпрограмма KbScan (программа 1)

Назначение: опрос матрицы 4x3 по столбцам (вызывается из обработчика INT0). Проходит Col1...Col4, в каждом читает строки и считает замкнутые контакты.

Входные данные: нет.

Выходные данные: A — код клавиши по табл. 3 T3, если нажата ровно одна; 0FFh — ни одной или несколько.

Используемые регистры: только A (PSW, B, DPTR, R2...R6 сохраняются в стеке).

3.2.2 Подпрограмма BufRead (программа 2)

Назначение: выдать самый старый отсчёт из кольцевого буфера (IDT7005, левый порт).

Входные данные: «хвост» TAIL_H:TAIL_L, «голова» HEAD_H:HEAD_L, флаг F_EMPTY.

Выходные данные: A — отсчёт; «хвост» сдвинут на ячейку (после BUF_END-1 — на BUF_START); F_OVF = 0; F_EMPTY = 1, если «хвост» догнал «голову». Пустой буфер: A = 0, указатели и флаги не меняются.

Используемые регистры: только A (PSW и DPTR сохраняются в стеке).

3.2.3 Подпрограмма Y2Out (программа 3)

Назначение: вычислить Y2, записать в регистр Y2 (74HC573) и выдать строб длительностью T2. $Y2 = (G + M + X1 + X2) \bmod 256$, если X1 не 0; при X1 = 0 — Y2 = 0.

Входные данные: X1, X2 (ОЗУ).

Выходные данные: значение в регистре Y2; PIN_Y2 = 0, Timer1 запущен на T2 (строб снимает его прерывание).

Используемые регистры: A (PSW и DPTR сохраняются в стеке).

3.2.4 Обработчик прерывания Timer1 (программа 3)

Назначение: Timer1 отсчитал T2 — конец строба Y2.

Распределение ресурсов микроконтроллера — прерывания, адреса внешних устройств и переменные — приведено в таблицах 3.1–3.3.

Таблица 3.1

Прерывания

Источник	Вектор	Назначение	Запуск
INT0 (P3.2)	0003h	нажатие клавиши	по спаду
Timer0	000Bh	отсчёт T_1 (205 мс) и T_3 (2380 мс), тик 5 мс	переполнение
INT1 (P3.3)	0013h	строб записи X_2	по спаду
Timer1	001Bh	отсчёт T_2 (1869 мкс)	переполнение
UART	0023h	не используется (заглушка)	—

Таблица 3.2

Адреса внешних устройств (обращение командами movx при P3.4 = 1)

Устройство	Выход дешифратора	Адреса
Индикатор	Y6	0C000h–0DFFFh
Клавиатура	Y7	0E000h–0FFFFh
Буфер (IDT7005)	Y4	8000h–9FFFh
Регистр Y2	Y5	0A000h–0BFFFh

Таблица 3.3

Переменные во внутреннем ОЗУ

Имя	Адрес	Назначение
HEAD_L, HEAD_H	44h, 45h	«голова» — адрес ячейки буфера для следующей записи
TAIL_L, TAIL_H	46h, 47h	«хвост» — адрес ячейки буфера для следующего чтения
F_EMPTY	20h.2 (бит 02h)	флаг пустоты буфера
F_OVF	21h.3 (бит 0Bh)	флаг переполнения буфера
X1	30h	сигнал X_1 — цифра с клавиатуры
X2	31h	сигнал X_2 — отсчёт из буфера

3.3. Связи программы с другими программами

Программа «Программа управления объектом» в ходе своей работы не использует других программ.

4. ВХОДНЫЕ ДАННЫЕ

4.1. Клавиатура

Клавиатура — матрица 4×3 (4 столбцов × 3 строк), сканирование по столбцам: в регистр столбцов (адрес 0E000h, разряды D0...D3) выдаётся 0 в опрашиваемый столбец, строки читаются через повторитель по тому же адресу (разряды D0...D2, 0 — контакт замкнут). Нажатие любой клавиши при всех столбцах в 0 вызывает прерывание INT0. Коды клавиш (табл. 3 ТЗ) и их расположение — в таблице 4.1.

Таблица 4.1

Расположение клавиш и их коды («клавиша» — код)

	Col1	Col2	Col3	Col4
Row1	«1» — 1	«2» — 2	«3» — 3	«Т» — 10
Row2	«4» — 4	«5» — 5	«6» — 6	«С» — 11
Row3	«7» — 7	«8» — 8	«9» — 9	«0» — 0

Результат опроса — код единственной нажатой клавиши (0–9 — цифры, 10 — «Тест», 11 — «Сброс») или 0FFh, если не нажато ничего или нажато больше одной клавиши. Цифра становится значением X_1 (начальное значение $X_1 = 0$).

4.2. Сигнал X2

X_2 — 8-разрядное целое без знака. Внешнее устройство записывает отсчёт в двухпортовое ОЗУ IDT7005 через свой порт и сопровождает запись стробом, который вызывает прерывание INT1. Отсчёты хранятся в кольцевом буфере на 1520 байт (адреса 8000h–85EFh).

5. ВЫХОДНЫЕ ДАННЫЕ

5.1. Характер и организация выходных данных

Индикация. На индикаторе отображается последняя введенная цифра — до ввода следующей, но не дольше $T_3 = 2,380$ с; при ошибке — код ошибки с горящей точкой др. Начальное состояние — индикатор погашен.

Кольцевой буфер во внешнем ОЗУ. Ячейки 8000h–85EFh. «Голова» — адрес ячейки для следующей записи, «хвост» — для следующего чтения; после последней ячейки следует первая. Буфер пуст — флаг пустоты = 1 (голова = хвост). Если буфер заполнен, новый отсчет затирает самый старый (хвост сдвигается) и устанавливается флаг переполнения; после чтения флаг переполнения сбрасывается.

Управляющие сигналы. Y_1 — строб низкого уровня длительностью 205 мс по клавише «Тест». $Y_2 = (9 + 2 + X_1 + X_2) \bmod 256$ при $X_1 \neq 0$, иначе $Y_2 = 0$ (пассивное состояние); вырабатывается, пока в буфере есть данные и $X_1 \neq 0$. Каждая запись Y_2 в выходной регистр сопровождается стробом длительностью 1869 мкс; до записи регистр выдает предыдущее значение.

Расчётные параметры работы программы. Константы таймеров (кварц 12 МГц, машинный цикл $T_{мц} = 12 / f = 1$ мкс, режим 1 — 16 бит):

ТН1:ТЛ1 = 65536 – $T_2 / T_{мц} = 65536 - 1869 = 63667 = 0F8B3h$ (строб Y_2 , таймер 1)

ТН0:ТЛ0 = 65536 – $5 \text{ мс} / T_{мц} = 65536 - 5000 = 60536 = 0EC78h$ (тик 5 мс, таймер 0)

$N_{T1} = T_1 / 5 \text{ мс} = 41$; $N_{T3} = T_3 / 5 \text{ мс} = 476$ (число тиков)

Минимальное время заполнения буфера (при $X_1 = 0$ сигнал Y_2 пассивен и буфер не читается, отсчеты поступают с максимальной интенсивностью):

$t_{\min} = V / f_{X2} = 1520 / 4 = 380,0$ с (6,3 мин).

Максимальная глубина стека для программы 1 — 13 байт (SP от 07h до 14h). Значение получено прогоном программы в симуляторе на всех сценариях проверки (каждая клавиша, одновременные нажатия, помеха на INT0) и складывается из адреса возврата при входе в обработчик INT0 (2 байта), адреса возврата при вызове процедуры опроса (2 байта) и сохраняемых в стеке регистров.

5.2. Формат, описание и способ кодирования выходных данных

Код символа (табл. 2 ТЗ) преобразуется в код сегментов по таблице 5.1: разряды D0...D7 регистра индикатора соответствуют сегментам a, b, c, d, e, f, g, dr; индикатор с общим анодом — сегмент горит при 0.

Таблица 5.1

Коды сегментов индикатора (dp g f e d c b a)			
Код	Символ	Двоичный	Шестн.
0	«0»	11000000	0C0h
1	«1»	11111001	0F9h
2	«2»	10100100	0A4h
3	«3»	10110000	0B0h
4	«4»	10011001	99h
5	«5»	10010010	92h
6	«6»	10000010	82h
7	«7»	11111000	0F8h
8	«8»	10000000	80h
9	«9»	10010000	90h
10	«E»	10000110	86h
11 и больше	погашен	11111111	0FFh

Таблица 5.2

Коды ошибок (символ с горящей точкой др, индикация до нажатия любой клавиши, но не дольше ТЗ)

Индикация	Смысл	Когда выставляется
«1.»	нажато несколько клавиш	опрос клавиатуры нашёл больше одного замкнутого контакта
«2.»	нажатие не распознано	прерывание пришло, а замкнутых контактов нет
«3.»	переполнение буфера	запись X_2 затёрла непрочитанный отсчёт
«4.»	повторный «Тест»	«Тест» нажат, пока идёт строб Y_1

В написанных программах ошибка клавиатуры обозначается одним кодом: процедура KbScan возвращает 0FFh и тогда, когда замкнуто несколько контактов (код «1.»), и когда прерывание пришло без нажатия (код «2.»); в полной системе их можно различить, если процедура будет отдельно сообщать число замкнутых контактов (оно уже считается в R3). Флаг переполнения буфера (код «3.») выставляет обработчик INT1, а процедура чтения его сбрасывает; проверка повторного «Теста» (код «4.») относится к полной системе и в сданные программы не входит. Реакция на любую ошибку одна — на индикатор выводится её код с точкой, и он гаснет при следующем нажатии или по истечении ТЗ.

Сигнал Y_2 выдаётся 8-разрядным кодом в выходной регистр (адрес 0A000h), строб — на выводе P1.3; сигнал Y_1 — на выводе P1.2.

В 16-разрядном режиме (режим 1) таймер 0 перезагружается в обработчике прерывания; задержка входа в прерывание (несколько машинных циклов на тик) компенсируется поправкой к загружаемому значению или учитывается как погрешность длительности. В режиме 2 (8 разрядов с автоперезагрузкой) перезагрузку выполняет сам таймер, а тик набирается программным счётчиком переполнений.

ПЕРЕЧЕНЬ ПРИНЯТЫХ СОКРАЩЕНИЙ

Таблица 5.3

Сокращения

Сокращение	Расшифровка
МК	микроконтроллер
МПС	микропроцессорная система
ОЗУ	оперативное запоминающее устройство
ПЗУ	постоянное запоминающее устройство
ЕСПД	единая система программной документации
ТЗ	техническое задание
CS	Chip Select, выбор микросхемы
IRQ	запрос прерывания
SP	указатель стека
DPTR	16-разрядный регистр-указатель данных
dp	десятичная точка индикатора

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

[illegible]

ПРИЛОЖЕНИЕ А

Листинги программ

Программа 1 — файл «Давыдова ЕП-код-1»

```
; Давыдова Е.П., А-09-23, 2, v1
$INCLUDE (vars.inc) ; адреса, биты, константы варианта; в файл для робота mpcode вклеит его текст
DSEG AT 30h
; переменных в ОЗУ нет: опрос работает на регистрах R2...R6 и B
CSEG AT 0h
    sjmp START
org 03h ; INT0 — нажатие клавиши: процедура вызывается отсюда
    call KbScan ; %proc%
    reti
org 0bh ; "заглушка" для Tmr0_ovf
    nop
    reti
org 13h ; "заглушка" для INT1
    nop
    reti
org 1bh ; "заглушка" для Tmr1_ovf
    nop
    reti
org 23h ; "заглушка" для UART
    nop
    reti
org 2bh
START:
; Инициализация МК *****
    mov SP, #07h
    setb PIN_CSEN ; разрешить дешифратор, чтобы дошёл CS клавиатуры
    mov DPTR, #ADR_KB
    clr A
    movx @DPTR, A ; все столбцы в 0: любое нажатие опустит строку и ~INT0
    setb IT0 ; INT0 — по спаду
    clr IE0 ; забыть спад, возможный при установке столбцов
    setb EX0 ; разрешить прерывание от клавиатуры
    setb EA ; общее разрешение прерываний
    jmp $ ; %stop%

; KbScan — опрос матрицы 4x3 по столбцам (вызывается из обработчика INT0).
; Проходит Col1...Col4, в каждом читает строки и считает замкнутые контакты.
; Вход: нет.
; Выход: A — код клавиши по табл. 3 ТЗ, если нажата ровно одна; 0FFh — ни одной или несколько.
; Портит: только A (PSW, B, DPTR, R2...R6 сохраняются в стеке).
KbScan:
    push PSW
    push B
    push DPL
    push DPH
    push AR2
    push AR3
    push AR4
    push AR5
    push AR6
    mov R2, #0FFh ; код найденной клавиши (пока нет)
    mov R3, #0 ; сколько контактов замкнуто
    mov R4, #0 ; номер столбца 0...3
    mov R5, #0FEh ; маска: 0 только в бите текущего столбца

KbCol:
    mov DPTR, #ADR_KB
    mov A, R5
    movx @DPTR, A ; опустить один столбец, остальные держать в 1
    movx A, @DPTR ; прочитать строки через 74HC244
    cpl A ; теперь 1 — строка замкнута
    anl A, #(1 SHL KB_ROWS) - 1 ; старшие линии шины к строкам не подключены
    mov B, A ; биты замкнутых строк этого столбца
    mov A, R4
    mov R6, A ; индекс в KeyTab: строка * KB_COLS + столбец, начинаем с Row1

KbRow:
    mov A, B
    jz KbNext ; в столбце больше нет замкнутых строк
    clr C
    rrc A ; младшая строка уходит в C
    mov B, A
    jnc KbSkip ; эта строка не замкнута
    inc R3 ; ещё один замкнутый контакт
    mov A, R6
    mov DPTR, #KeyTab
    movc A, @A+DPTR ; код клавиши по раскладке рис. 4
    mov R2, A

KbSkip:
```

```

mov A, R6
add A, #KB_COLS      ; та же колонка, следующая строка
mov R6, A
sjmp KbRow

KbNext:
mov A, R5
rl A                  ; ноль переезжает на следующий столбец
mov R5, A
inc R4
cjne R4, #KB_COLS, KbCol
mov DPTR, #ADR_KB
clr A
movx @DPTR, A        ; снова все столбцы в 0 – ждём следующее нажатие
clr IE0              ; спады ~INT0 во время перебора – не новое нажатие
mov A, R2
cjne R3, #1, KbBad   ; ровно одна клавиша – код, иначе ошибка
sjmp KbOut

KbBad:
mov A, #0FFh         ; ничего не нажато (помеха) или нажато несколько

KbOut:
pop AR6
pop AR5
pop AR4
pop AR3
pop AR2
pop DPH
pop DPL
pop B
pop PSW
ret

; KeyTab – коды клавиш (табл. 3 T3) в порядке строка * 4 + столбец, раскладка 4x3 рис. 4:
; Row1: 1 2 3 T, Row2: 4 5 6 C, Row3: 7 8 9 0; T = 10, C = 11.
KeyTab: DB 1, 2, 3, 10
        DB 4, 5, 6, 11
        DB 7, 8, 9, 0

END

```

Программа 2 — файл «Давыдова ЕП-код-2»

```

; Давыдова Е.П., А-09-23, 2, v1
$INCLUDE (vars.inc)      ; адреса, биты, константы варианта; в файл для работа mpcode вклеит его текст
DSEG AT 30h
; своих переменных нет: указатели и флаги – по адресам из vars.inc
CSEG AT 0h
sjmp START

org 03h ; "заглушка" для INT0
nop
reti

org 0bh ; "заглушка" для Tmr0_ovf
nop
reti

org 13h ; "заглушка" для INT1
nop
reti

org 1bh ; "заглушка" для Tmr1_ovf
nop
reti

org 23h ; "заглушка" для UART
nop
reti

org 2bh
START:
; Инициализация МК *****
mov SP, #07h
setb PIN_CSEN      ; дешифратор включён – CS буфера будет работать
mov HEAD_L, #LOW(BUF_START) ; запись пойдёт в первую ячейку буфера
mov HEAD_H, #HIGH(BUF_START)
mov TAIL_L, #LOW(BUF_START) ; читать тоже начнём с первой ячейки
mov TAIL_H, #HIGH(BUF_START)
setb F_EMPTY      ; буфер пуст
clr F_OVF         ; переполнения не было
call BufRead ; %proc%
jmp $ ; %stop%

; BufRead – выдать самый старый отсчёт из кольцевого буфера (IDT7005, левый порт).
; Вход: «хвост» TAIL_H:TAIL_L, «голова» HEAD_H:HEAD_L, флаг F_EMPTY.
; Выход: A – отсчёт; «хвост» сдвинут на ячейку (после BUF_END-1 – на BUF_START);
; F_OVF = 0; F_EMPTY = 1, если «хвост» догнал «голову».
; Пустой буфер: A = 0, указатели и флаги не меняются.
; Портит: только A (PSW и DPTR сохраняются в стеке).
BufRead:
jb F_EMPTY, BrNone ; читать нечего
push PSW
push DPL

```

```

push DPH
mov DPL, TAIL_L
mov DPH, TAIL_H
movx A, @DPTR          ; самый старый отсчёт
push ACC               ; сберечь его, пока двигаем «хвост»
inc DPTR               ; следующая ячейка
mov A, DPL
cjne A, #LOW(BUF_END), BrKeep ; ещё не вышли за буфер
mov A, DPH
cjne A, #HIGH(BUF_END), BrKeep
mov DPTR, #BUF_START   ; вышли за последнюю ячейку — на начало кольца
BrKeep:
mov TAIL_L, DPL
mov TAIL_H, DPH
clr F_OVF              ; одна ячейка освободилась — буфер не полон
mov A, TAIL_L
cjne A, HEAD_L, BrDone ; «хвост» не совпал с «головой» — данные ещё есть
mov A, TAIL_H
cjne A, HEAD_H, BrDone
setb F_EMPTY           ; прочитан последний отсчёт
BrDone:
pop ACC                ; вернуть отсчёт в A
pop DPH
pop DPL
pop PSW
ret
BrNone:
clr A                  ; пусто: отдать 0, ничего не трогать
ret
END

```

Программа 3 — файл «Давыдова ЕП-код-3»

```

; Давыдова Е.П., А-09-23, 2, v1
$INCLUDE (vars.inc)      ; адреса, биты, константы варианта; в файл для работа mpcode вклеит его текст
DSEG AT 30h
X1: DS 1                  ; цифра с клавиатуры (0 — Y2 пассивен)
X2: DS 1                  ; отсчёт из буфера
CSEG AT 0h
sjmp START
org 03h ; "заглушка" для INT0
nop
reti
org 0bh ; "заглушка" для Tmr0_ovf
nop
reti
org 13h ; "заглушка" для INT1
nop
reti
org 1bh ; Timer1 отсчитал T2 — конец строба Y2
; таймер больше не нужен до следующего вывода
; строб в пассивную 1
clr TR1
setb PIN_Y2
reti
org 23h ; "заглушка" для UART
nop
reti
org 2bh
START:
; Инициализация МК *****
mov SP, #07h
setb PIN_CSEN           ; дешифратор включён — дойдёт CS регистра Y2
setb PIN_Y2             ; строб Y2 пассивен
mov TMOD, #10h          ; Timer1 — режим 1 (16 бит), Timer0 не используется
setb ET1                ; прерывание по переполнению Timer1
setb EA
mov X1, #3               ; пример входных данных
mov X2, #200
call Y2Out ; %proc%
jmp $ ; %stop%

; Y2Out — вычислить Y2, записать в регистр Y2 (74НС573) и выдать строб длительностью T2.
; Y2 = (G + M + X1 + X2) mod 256, если X1 не 0; при X1 = 0 — Y2 = 0.
; Вход: X1, X2 (03У).
; Выход: значение в регистре Y2; PIN_Y2 = 0, Timer1 запущен на T2 (строб снимает его прерывание).
; Портит: A (PSW и DPTR сохраняются в стеке).
Y2Out:
push PSW
push DPL
push DPH
mov A, X1
jz Y2Put                ; X1 = 0 — выводим 0
add A, X2                ; перенос отбрасывается: это и есть mod 256
add A, #Y2_CONST         ; + (G + M)
Y2Put:
mov DPTR, #ADR_Y2

```

```
movx @DPTR, A      ; сначала значение в регистр
clr TR1            ; на случай незавершённого прошлого строба
mov TH1, #T2_RELOAD_H ; Timer1 отсчитает T2
mov TL1, #T2_RELOAD_L
clr TF1            ; убрать старое переполнение
clr PIN_Y2         ; начало строба
setb TR1           ; пошёл отсчёт T2
pop DPH
pop DPL
pop PSW
ret
END
```